PRACTICAL NO : 2

Name : Ghanwat Payal Ganesh Roll No : 222 Class : SE (AIML) Batch : S1

Problem Statement: Stack : CO1, CO2 I

Implement stack as an abstract data type using singly linked list and use this ADT for conversion of infix expression to postfix, prefix.

Code :

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

// Node for Stack
struct Node
{
 char data;
 Node *next;
};

// Stack ADT
class Stack
{
 Node *top;

public:
 Stack()
{
 top = NULL;
}
```

```
void push(char x)
{
 Node *temp = new Node;
 temp->data = x;
 temp->next = top;
 top = temp;
}

char pop()
{
 if (top == NULL)
   return '\0';

 Node *temp = top;
 char x = temp->data;
 top = top->next;
 delete temp;
 return x;
}

char peek()
{
 if (top == NULL)
 return '\0';
 return top->data;
}
```

```
bool isEmpty()
{
 return top == NULL;
 }
};

// Function to check precedence
int precedence(char op)
{
 if (op == '^')
   return 3;
 if (op == '*' || op == '/')
   return 2;
 if (op == '+' || op == '-')
   return 1;
 return 0;
}

// Infix to Postfix
string infixToPostfix(string infix)
{
 Stack s;
 string postfix = "";

 for (char ch : infix)
{
 if (isalnum(ch))
 {
```

```
  postfix += ch;
 }
 else if (ch == '(')
 {
  s.push(ch);
 }
 else if (ch == ')')
 {
  while (!s.isEmpty() && s.peek() != '(')
    postfix += s.pop();
 s.pop();
 }
  else
  {
   while (!s.isEmpty() && precedence(s.peek()) >= precedence(ch))
      postfix += s.pop();

  s.push(ch);
  }
}
while (!s.isEmpty())
   postfix += s.pop();
return postfix;
}

// Infix to Prefix
string infixToPrefix(string infix)
{
```

```
 reverse(infix.begin(), infix.end());

 for (int i = 0; i < infix.length(); i++)
 {
  if (infix[i] == '(')
      infix[i] = ')';
  else if (infix[i] == ')')
      infix[i] = '(';
}

string prefix = infixToPostfix(infix);

reverse(prefix.begin(), prefix.end());

return prefix;
}

int main()
{
 string infix;

 cout << "Enter Infix Expression: ";
 cin >> infix;

 cout << "\nPostfix Expression : "
  << infixToPostfix(infix);

 cout << "\nPrefix Expression : "
```

```
  << infixToPrefix(infix);
return 0;
}
```

Output:

Output 1

Input

Enter Infix Expression: A+B/C

Output

Postfix Expression : ABC/+

Prefix Expression : +A/BC

=== Code Execution Successful ===